

TRAX ILIAS BRIDGE

Documentation développeur détaillée de l'adaptateur

Analyse détaillée de aggregate.php, apache-trax-ilias-adapter.conf et config.php

Version documentée	2.0.3
Date	16/06/2026
Audience	Développeurs, mainteneurs, intégrateurs avancés
Périmètre	ILIAS 10 + Trax LRS 3 - cmi5, Learning Experiences, Ranking
Fichiers étudiés	aggregate.php, apache-trax-ilias-adapter.conf, config.sample.php

Document rédigé à partir de l'archive source v2.0.3 fournie et du comportement validé : cmi5 OK, Learning Experiences OK, Ranking OK, xAPI non-cmi5 non perturbé.

Table des matières

- 1. Vue d'ensemble du code**
- 2. apache-trax-iliad-adapter.conf**
- 3. config.php**
- 4. aggregate.php - initialisation et routage**
- 5. Flux cmi5 tokens**
- 6. Flux statements aggregate**
- 7. Moteur d'agrégation ILIAS**
- 8. Gestion des chemins et filtres**
- 9. HTTP client et relais de réponse**
- 10. Sécurité du code**
- 11. Limites techniques**
- 12. Bonnes pratiques d'évolution**
- 13. Stratégie de test développeur**
- 14. Annexes code**

Les titres du document utilisent les styles Word afin de pouvoir générer ou mettre à jour une table des matières automatique dans Word si nécessaire.

1. Vue d'ensemble du code

La version 2.0.3 conserve le nom historique `aggregate.php`, mais ce fichier est en réalité le front controller complet de l'adaptateur. Il route les requêtes interceptées par Apache vers deux traitements : `proxy cmi5/tokens` et `compatibilité statements/aggregate`.

Fichier	Nature	Responsabilité développeur
<code>aggregate.php</code>	Script PHP frontal	Router les requêtes, nettoyer <code>activity_id</code> , lire Trax, appliquer une pipeline simplifiée, relayer les réponses.
<code>apache-trax-ilias-adapter.conf</code>	Configuration Apache	Définir précisément les URL interceptées avant Trax/Laravel.
<code>config.php</code>	Configuration runtime	Définir URL cible Trax, résolution client/store, limite de lecture et debug.

Constantes principales

```
const BRIDGE_VERSION = '2.0.3';
const BYPASS_HEADER = 'X-Trax-Ilias-Bridge-Bypass: 1';
```

`BRIDGE_VERSION` sert à identifier la version dans les réponses JSON et les requêtes sortantes. `BYPASS_HEADER` marque les appels HTTP que l'adaptateur effectue lui-même vers Trax.

2. apache-trax-ilias-adapter.conf

2.1 Objectif

Le fichier Apache est la porte d'entrée de l'adaptateur. Sans lui, les URL `/cmi5/tokens` et `/api/statements/aggregate` seraient traitées par Trax/Laravel ou retourneraient une erreur non compatible avec ILIAS.

2.2 Pourquoi AliasMatch

La v2.0.3 remplace les `RewriteRule` par `AliasMatch`. Le commentaire du fichier indique que, dans certaines installations Apache/Trax, les `RewriteRule` en contexte serveur ne sont pas appliquées avant Laravel. `AliasMatch` force la résolution de l'URL vers le script PHP de l'adaptateur.

2.3 Les trois routes

Route Apache	Script appelé	Usage
<code>^/trax/api/gateway/clients/[^]+/stores/api/statements/aggregate\$</code>	<code>aggregate.php</code>	Reporting ILIAS lorsque l'URL perd le store.
<code>^/trax/api/gateway/clients/[^]+/stores/[^]+/api/statements/aggregate\$</code>	<code>aggregate.php</code>	Reporting ILIAS lorsque l'URL contient le store.
<code>^/trax/api/gateway/clients/[^]+/stores/[^]+/cmi5/tokens\$</code>	<code>aggregate.php</code>	Pré-lancement cmi5.

2.4 Directory et Authorization

Le bloc `Directory` autorise l'accès au répertoire de l'adaptateur, désactive l'indexation et interdit les `.htaccess`. La directive `SetEnvIf Authorization` permet à PHP de récupérer l'en-tête `Authorization` selon la manière dont Apache expose cet en-tête.

```
<Directory "/var/www/trax-ilias-aggregate-adapter">
    Require all granted
    Options -Indexes
    AllowOverride None
</Directory>

SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1
```

2.5 Point de vigilance développeur

AliasMatch est volontairement étroit

Ne pas élargir les expressions sans test de non-régression. En particulier, /xapi/statements doit rester direct vers Trax afin de ne pas perturber les traces xAPI non-cmi5.

3. config.php

config.php retourne un tableau PHP. Le script charge config.php s'il existe, sinon config.sample.php. Le sample ne doit pas contenir de secret ; la configuration réelle peut être ignorée par Git.

3.1 Chargement

```
$configFile = __DIR__ . '/config.php';
$config = is_file($configFile) ? require $configFile : require __DIR__ . '/config.sample.php';
```

3.2 Paramètres détaillés

Clé	Type	Utilisée par	Détail
trax_base_url	string null	buildBaseUrl()	URL racine Trax cible. Si null, le script reconstruit scheme + host depuis la requête entrante.
aggregate_store_strategy	string	resolveStore()	client_name signifie client = store lorsque l'URL aggregate ne contient pas le store.
client_store_map	array	resolveStore()	Mapping explicite prioritaire client Trax vers store Trax.
default_store	string	resolveStore()	Store de repli lorsque la stratégie client_name n'est pas utilisée.
max_statements_to_fetch	int	handleStatementsAggregate()	Limite passée à /xapi/statements?limit=...
debug	bool	catch global	Affiche message et trace détaillée en cas d'exception.

3.3 Résolution du store

Le cas important est l'URL ILIAS /clients/{client}/stores/api/statements/aggregate : le nom du store a disparu. La fonction resolveStore() applique l'ordre suivant : client_store_map, stratégie client_name, default_store, exception.

```
Ordre de résolution :
1. client_store_map[client] si défini
2. aggregate_store_strategy == client_name => store = client
3. default_store si renseigné
4. RuntimeException si aucun store ne peut être résolu
```

4. aggregate.php - initialisation et routage

4.1 Initialisation

Le script active strict_types, charge la configuration, lit REQUEST_URI, extrait le path avec parse_url, normalise la méthode HTTP, puis route via trois expressions régulières.

```
$requestUri = $_SERVER['REQUEST_URI'] ?? '/';
$path = parse_url($requestUri, PHP_URL_PATH) ?: '/';
$method = strtoupper($_SERVER['REQUEST_METHOD'] ?? 'GET');
```

4.2 Routage

```
if (preg_match('#^/trax/api/gateway/clients/([^\s]+)/stores/([^\s]+)/cmi5/tokens$', $path, $m)) {
    validateName($m[1], 'client');
    validateName($m[2], 'store');
    handleCmi5Tokens($config, $method, $path, $m[1], $m[2]);
    exit;
}
```

Ordre	Regex	Handler
1	/clients/{client}/stores/{store}/cmi5/tokens	handleCmi5Tokens(\$config, \$method, \$path, \$client, \$store)
2	/clients/{client}/stores/{store}/api/statements/aggregate	handleStatementsAggregate(\$config, \$method, \$client, \$store)
3	/clients/{client}/stores/api/statements/aggregate	handleStatementsAggregate(\$config, \$method, \$client, null)
4	Autre route	jsonResponse(404, route_not_handled)

4.3 Validation des noms

validateName() limite client et store aux caractères A-Z, a-z, 0-9, underscore, point et tiret. Cela évite les noms contenant des slashes, caractères d'échappement ou fragments de chemin.

```
function validateName(string $value, string $label): void
{
    if (!preg_match('/^[A-Za-z0-9_.-]+$/ ', $value)) {
        throw new InvalidArgumentException("Invalid $label name.");
    }
}
```

5. Flux cmi5 tokens

5.1 handleCmi5Tokens()

Cette fonction agit comme un proxy correcteur. Elle ne calcule pas de token elle-même ; elle supprime seulement le paramètre incompatible activity_id puis appelle le vrai endpoint Trax.

```
$queryParams = $_GET;
unset($queryParams['activity_id']);

$rawBody = file_get_contents('php://input') ?: '';
$contentType = $_SERVER['CONTENT_TYPE'] ?? $_SERVER['HTTP_CONTENT_TYPE'] ?? '';
$body = cleanRequestBody($rawBody, $contentType);
```

5.2 Nettoyage du corps de requête

cleanRequestBody() supporte trois cas : corps vide, JSON, formulaire application/x-www-form-urlencoded et fallback conservateur. Dans chaque cas structuré, activity_id est supprimé si présent.

Content-Type	Traitement
application/json	json_decode en tableau, unset activity_id, json_encode sans échapper Unicode/slashes.
application/x-www-form-urlencoded	parse_str, unset activity_id, http_build_query.
autre texte parsable	parse_str en fallback ; si activity_id existe, reconstruction sans activity_id.
autre non parsable	Le corps original est conservé.

5.3 Transmission à Trax

```
$targetUrl = buildBaseUrl($config) . $path;
if ($queryParams !== []) {
    $targetUrl .= '?' . http_build_query($queryParams);
}

$headers = forwardHeaders(['authorization', 'accept', 'content-type', 'x-experience-api-version']);
$headers[] = BYPASS_HEADER;
$headers[] = 'X-Trax-Ilias-Bridge-Version: ' . BRIDGE_VERSION;

$response = httpRequest($method, $targetUrl, $headers, $body);
relayResponse($response);
```

Le statut HTTP et le corps de réponse Trax sont relayés à ILIAS. Le script ne reformate pas une réponse cmi5 réussie ou échouée.

6. Flux statements aggregate

6.1 Méthode acceptée

handleStatementsAggregate() accepte uniquement GET. Toute autre méthode renvoie 405 method_not_allowed.

```
if ($method !== 'GET') {
    jsonResponse(405, ['error' => 'method_not_allowed', 'allowed' => ['GET']]);
    return;
}
```

6.2 Lecture Trax

Le handler résout le store, construit une URL /xapi/statements avec limit, puis force X-Experience-API-Version à 1.0.3 pour éviter que Trax reçoive deux valeurs identiques si ILIAS en avait déjà envoyé une.

```
$targetUrl = buildBaseUrl($config)
    . '/trax/api/gateway/clients/' . rawurlencode($client)
    . '/stores/' . rawurlencode($store)
    . '/xapi/statements?limit=' . $limit;

$headers = forwardHeaders(['authorization', 'accept']);
$headers[] = BYPASS_HEADER;
$headers[] = 'X-Trax-Ilias-Bridge-Version: ' . BRIDGE_VERSION;
$headers[] = 'X-Experience-API-Version: 1.0.3';
```

6.3 Pipeline ILIAS

La pipeline arrive dans le paramètre GET pipeline, encodée en JSON. Si elle est absente ou invalide, le script utilise une pipeline vide. La réponse de Trax doit contenir un tableau statements.

```
$pipelineRaw = $_GET['pipeline'] ?? '[]';
$pipeline = json_decode((string)$pipelineRaw, true);
if (!is_array($pipeline)) {
    $pipeline = [];
}

$result = aggregateForIlias($statements, $pipeline);
jsonResponse(200, $result);
```

7. Moteur d'agrégation ILIAS

aggregateForIlias() transforme chaque statement xAPI en document double forme : un champ statement contenant le statement complet, et les champs xAPI au premier niveau. Cette double forme permet de satisfaire des pipelines qui utilisent soit statement.verb.id, soit verb.id.

```
$docs[] = ['statement' => $s] + $s;
```

7.1 Stages supportés

Stage	Fonction	Usage ILIAS
\$match	Filtrage par critères.	Limiter aux statements de l'objet, de l'acteur, du verbe ou d'une période.
\$sort	Tri multi-champs.	Classer par date, score ou ordre demandé.
\$skip	Décalage.	Pagination.
\$limit	Limite.	Pagination ou top N.
\$group	Regroupement et accumulateurs.	Verbes distincts, lignes Ranking par participant.
\$facet	Réponse maxcount + statements.	Tableau Learning Experiences avec total et liste.

7.2 Sort

sortDocs() compare successivement les champs déclarés dans la map de tri. La direction négative inverse la comparaison. Les valeurs sont obtenues avec pathValues() puis firstValue().

7.3 Group

groupDocs() évalue _id pour chaque document, ignore les identifiants vides, conserve l'ordre d'apparition des groupes, puis calcule les champs demandés. Cette fonction contient une protection spécifique pour le Ranking : si account est absent mais que le regroupement porte sur actor.account.name, account reçoit _id comme fallback.

```
if (!array_key_exists('account', $row) && is_string($idExpression) && str_contains($idExpression,
'actor.account.name')) {
    $row['account'] = is_scalar($row['_id']) ? (string)$row['_id'] : '';
}
```

7.4 Accumulateurs supportés

Accumulateur	Comportement implémenté
\$last	Évalue l'expression sur le dernier document du groupe.
\$first	Évalue l'expression sur le premier document du groupe.
\$push	Collecte les valeurs non nulles et non vides.
\$max	Retourne le maximum des valeurs non nulles.
\$min	Retourne le minimum des valeurs non nulles.
\$sum	Additionne l'argument numérique ou les valeurs numériques évaluées.
opérateur inconnu	Évalue l'expression sur le dernier document, comportement permissif.

7.5 Facet

Le support de \$facet est volontairement minimal. Dès qu'un stage \$facet est rencontré, le script renvoie un tableau contenant un objet avec maxcount et statements. Ce format correspond aux attentes habituelles du tableau ILIAS.

```
return [[
    'maxcount' => $total,
```

```
'statements' => array_values($docs),
[]];
```

8. Gestion des chemins et filtres

8.1 pathValues()

pathValues() permet d'extraire une ou plusieurs valeurs depuis un chemin pointé, par exemple statement.verb.id ou actor.account.name. La fonction descend dans les tableaux associatifs et sait traverser les listes pour collecter plusieurs valeurs.

8.2 matchesCriteria()

matchesCriteria() applique un document à une map de critères. Elle supporte \$and et \$or au premier niveau, puis délègue chaque condition à conditionMatches().

8.3 Opérateurs de condition

Opérateur	Sens
\$exists	Vérifie la présence ou l'absence d'une valeur.
\$in	Au moins une valeur du document appartient à la liste attendue.
\$nin	Aucune valeur du document ne doit appartenir à la liste.
\$ne	Aucune valeur ne doit être égale à la valeur attendue.
\$eq	Au moins une valeur doit être égale à la valeur attendue.
\$gte / \$gt	Comparaison supérieure ou supérieure/égale.
\$lte / \$lt	Comparaison inférieure ou inférieure/égale.
\$regex	Expression régulière insensible à la casse.
opérateur inconnu	Ignoré de manière permissive pour ne pas casser l'interface ILIAS.

Choix de conception

Le moteur préfère être permissif sur les opérateurs inconnus afin d'éviter de bloquer l'interface ILIAS. Cette stratégie est adaptée à un adaptateur de compatibilité, mais elle doit être documentée lors des évolutions.

9. HTTP client et relais de réponse

9.1 buildBaseUrl()

buildBaseUrl() retourne trax_base_url si configuré. Sinon elle reconstruit scheme://host à partir de HTTPS, HTTP_X_FORWARDED_PROTO et HTTP_HOST.

9.2 forwardHeaders()

forwardHeaders() parcourt \$_SERVER, sélectionne uniquement les en-têtes autorisés, normalise le nom et reconstitue Authorization si nécessaire à partir de REDIRECT_HTTP_AUTHORIZATION ou PHP_AUTH_USER/PHP_AUTH_PW.

9.3 httpRequest()

httpRequest() utilise cURL en mode RETURNTRANSFER + HEADER. Le timeout de connexion est de 10 secondes et le timeout total de 60 secondes. Les redirections ne sont pas suivies.

Option cURL	Valeur	Impact
CURLOPT_CUSTOMREQUEST	Méthode entrante ou GET	Conserve le type d'appel.
CURLOPT_HTTPHEADER	Headers filtrés + headers adaptateur	Transmet auth et version.
CURLOPT_RETURNTRANSFER	true	Récupère corps + headers.
CURLOPT_HEADER	true	Permet de séparer headers et body.

Option cURL	Valeur	Impact
CURLOPT_FOLLOWLOCATION	false	Évite de suivre automatiquement une redirection.
CURLOPT_CONNECTTIMEOUT	10	Limite le temps de connexion.
CURLOPT_TIMEOUT	60	Limite la durée totale.

9.4 relayResponse()

relayResponse() renvoie le statut Trax et relaie les en-têtes sauf ceux qui ne doivent pas être dupliqués ou imposés par PHP/Apache : transfer-encoding, content-length, connection, server et date.

10. Sécurité du code

- Validation stricte des noms client/store via validateName().
- Pas de secrets Trax en dur dans aggregate.php.
- Filtrage des en-têtes relayés : seuls les en-têtes explicitement autorisés sont transmis.
- debug désactivable pour masquer les traces d'exception en production.
- Aucune interception de /xapi/statements dans la conf livrée.
- Corps cmi5 nettoyé uniquement pour supprimer activity_id ; activityId est conservé.

Risques à surveiller

- Une règle Apache trop large peut détourner les traces xAPI standard.
- Une URL trax_base_url pointant vers une route ré-interceptée peut provoquer une boucle selon l'architecture Apache.
- Une valeur max_statements_to_fetch très élevée peut provoquer un coût mémoire important.
- Le mode debug peut exposer des chemins et traces techniques.

11. Limites techniques

Limite	Conséquence	Approche recommandée
Agrégation partielle	Tous les opérateurs MongoDB ne sont pas implémentés.	Ajouter seulement les opérateurs réellement nécessaires aux pipelines ILIAS observées.
Lecture limitée par limit	Reporting incomplet si la volumétrie dépasse la limite.	Ajuster max_statements_to_fetch ou ajouter une pagination Trax côté adaptateur si nécessaire.
Pas de persistance locale	Aucune cache ni indexation propre.	Surveiller les performances Trax et PHP.
Dépendance au format ILIAS	Une évolution d'ILIAS peut changer la pipeline.	Logger les pipelines en environnement de test et ajouter des tests.

12. Bonnes pratiques d'évolution

- Conserver aggregate.php comme front controller tant que le dépôt garde la compatibilité de noms.
- Ne pas ajouter d'interception Apache sans test xAPI non-cmi5.
- Ajouter un opérateur d'agrégation seulement après avoir capturé une pipeline ILIAS réelle qui le nécessite.
- Documenter chaque changement dans README.md et dans un CHANGELOG.
- Ajouter une variable de configuration plutôt qu'une valeur codée en dur lorsque le comportement dépend de l'environnement.
- Préserver la compatibilité avec PHP 8.1+ si l'environnement cible n'est pas uniformément en PHP 8.2.
- Éviter d'exposer les credentials dans les logs de debug.

Ajout d'un nouvel opérateur conditionnel

1. Ajouter un case dans conditionMatches().
2. Écrire un test avec un document simple et une pipeline \$match.
3. Vérifier que les opérateurs inconnus restent permissifs si c'est le comportement souhaité.
4. Documenter l'opérateur dans la documentation développeur.

Ajout d'un nouvel accumulateur

5. Ajouter un case dans evaluateAccumulator().
6. Gérer le cas groupe vide.
7. Définir comment traiter les valeurs nulles, vides ou non numériques.
8. Tester avec une pipeline de Ranking réelle ou simulée.

13. Stratégie de test développeur

13.1 Tests manuels minimum

Test	Objectif
GET /xapi/about	Vérifier que Trax est joignable avec les credentials.
GET /stores/api/statements/aggregate pipeline group verb	Vérifier la route aggregate sans store.
GET /stores/{store}/api/statements/aggregate pipeline group verb	Vérifier la route aggregate avec store.
Lancement cmi5 ILIAS	Valider suppression activity_id et token.
Ressource xAPI non-cmi5	Valider absence d'interception /xapi/statements.
Ranking ILIAS	Valider account, score, timestamp, duration.

13.2 Tests de non-régression recommandés

- Pipeline \$match simple sur statement.object.id.
- Pipeline \$group par statement.verb.id.
- Pipeline \$group par actor.account.name avec \$last score/date/duration.
- Pipeline \$facet retournant maxcount et statements.
- Corps cmi5 JSON contenant activity_id et activityId.
- Corps cmi5 form-urlencoded contenant activity_id et activityId.
- Authorization présente via HTTP_AUTHORIZATION, REDIRECT_HTTP_AUTHORIZATION et PHP_AUTH_USER.

```
# Exemple de surveillance pendant les tests :
tail -f /var/log/httpd/access_log | grep -E "xapi/statements|cmi5/tokens|statements/aggregate"

grep -iE "cmix|ranking|account|LRS error|exception|error|500" /var/www/logs/ilias.log
```

14. Annexes code

14.1 apache-trax-ilias-adapter.conf complet

```
# Trax ILIAS Bridge - Apache configuration
#
# Version 2.0.3
# Configuration volontairement basée sur AliasMatch : dans certaines installations
# Apache/Trax, les RewriteRule en contexte serveur ne sont pas appliquées avant
# Laravel. AliasMatch force l'interception par l'adaptateur.
#
# Noms conservés par rapport à la première version :
# - apache-trax-ilias-adapter.conf
# - aggregate.php
# - /var/www/trax-ilias-aggregate-adapter

# 1) Learning Experiences / Ranking avec URL native ILIAS 10 qui perd le store :
# /clients/{client}/stores/api/statements/aggregate
AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/api/statements/aggregate$" "/var/www/trax-ilias-aggregate-adapter/aggregate.php"

# 2) Learning Experiences / Ranking avec URL contenant le store :
# /clients/{client}/stores/{store}/api/statements/aggregate
```

```
AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/[^/]+/api/statements/aggregate$" "/var/www/trax-ili-as-
aggregate-adapter/aggregate.php"

# 3) Pré-lancement cmi5 :
# ILIAS 10 envoie activity_id ; l'adaptateur le supprime puis transmet à Trax.
AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/[^/]+/cmi5/tokens$" "/var/www/trax-ili-as-aggregate-
adapter/aggregate.php"

<Directory "/var/www/trax-ili-as-aggregate-adapter">
    Require all granted
    Options -Indexes
    AllowOverride None
</Directory>

# Permet à PHP de récupérer l'en-tête Authorization si nécessaire.
SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1
```

14.2 config.sample.php complet

```
<?php
/**
 * Configuration de l'adaptateur Trax ILIAS Bridge.
 */
return [
    /**
     * null = utilise automatiquement le protocole et l'hôte de la requête entrante.
     * Exemple possible si Trax est joignable localement : 'http://127.0.0.1'
     *
     * L'adaptateur ajoute l'en-tête X-Trax-Ilias-Bridge-Bypass pour éviter les
     * boucles de réécriture Apache quand cette URL pointe vers le même vhost.
     */
    'trax_base_url' => null,

    /**
     * ILIAS 10 construit souvent l'URL d'agrégation ainsi :
     * /trax/api/gateway/clients/{client}/stores/api/statements/aggregate
     * Le nom du store est donc perdu.
     *
     * Stratégie par défaut : client_name
     * Exemple : client ppm -> store ppm.
     *
     * Pour les exceptions, utiliser client_store_map.
     */
    'aggregate_store_strategy' => 'client_name',

    /**
     * Correspondance client Trax -> store Trax.
     * Exemple :
     * 'ppm' => 'ppm',
     * 'eform' => 'default',
     */
    'client_store_map' => [
        // 'ppm' => 'ppm',
        // 'eform' => 'default',
    ],

    /**
     * Utilisé seulement si aggregate_store_strategy n'est pas client_name
     * et si le client n'est pas déclaré dans client_store_map.
     */
]
```

```

'default_store' => 'default',

/**
 * Nombre maximum de statements xAPI chargés depuis Trax pour construire
 * la réponse attendue par ILIAS Learning Experiences / Ranking.
 */
'max_statements_to_fetch' => 5000,

/**
 * true = erreurs détaillées en JSON.
 * Laisser false en production.
 */
'debug' => false,
];

```

14.3 Index des fonctions aggregate.php

Fonction	Rôle
handleCmi5Tokens	Proxy cmi5, suppression activity_id, relais vers Trax.
handleStatementsAggregate	Endpoint de compatibilité aggregate, lecture /xapi/statements, application pipeline.
cleanRequestBody	Nettoyage des corps JSON/form-urlencoded pour supprimer activity_id.
aggregateForIlias	Moteur principal d'application des stages de pipeline.
sortDocs	Tri des documents selon la map \$sort.
groupDocs	Regroupement par _id et calcul des champs agrégés.
evaluateAccumulator	Implémentation \$last, \$first, \$push, \$max, \$min, \$sum.
evaluateExpression	Évalue les expressions de type \$chemin ou retourne la valeur littérale.
matchesCriteria	Applique \$match, \$and, \$or et conditions de champ.
conditionMatches	Évalue les opérateurs de condition.
pathValues / pathValuesRecursive	Extrait des valeurs depuis un chemin pointé, y compris dans les listes.
resolveStore	Résout le store depuis client_store_map, stratégie ou default_store.
buildBaseUrl	Construit l'URL de base Trax cible.
forwardHeaders	Filtre et reconstitue les en-têtes à transmettre.
httpRequest	Effectue l'appel cURL sortant.
parseResponseHeaders	Transforme les headers bruts cURL en tableau.
relayResponse	Relaie statut, headers filtrés et corps vers ILIAS.
jsonResponse	Produit une réponse JSON contrôlée.
validateName	Valide les noms client/store.

14.4 Extrait complet du routage aggregate.php

```

try {
    $requestUri = $_SERVER['REQUEST_URI'] ?? '/';
    $path = parse_url($requestUri, PHP_URL_PATH) ?: '/';
    $method = strtoupper($_SERVER['REQUEST_METHOD'] ?? 'GET');

    if (preg_match('#^/trax/api/gateway/clients/([^\s]+)/stores/([^\s]+)/cmi5/tokens$', $path, $m)) {
        validateName($m[1], 'client');
        validateName($m[2], 'store');
        handleCmi5Tokens($config, $method, $path, $m[1], $m[2]);
        exit;
    }
}

```

```

    }

    if (preg_match('#^/trax/api/gateway/clients/([^\s]+)/stores/([^\s]+)/api/statements/aggregate$#', $path, $m))
    {
        validateName($m[1], 'client');
        validateName($m[2], 'store');
        handleStatementsAggregate($config, $method, $m[1], $m[2]);
        exit;
    }

    if (preg_match('#^/trax/api/gateway/clients/([^\s]+)/stores/api/statements/aggregate$#', $path, $m)) {
        validateName($m[1], 'client');
        handleStatementsAggregate($config, $method, $m[1], null);
        exit;
    }

    jsonResponse(404, [
        'error' => 'route_not_handled',
        'message' => 'This URL is not handled by Trax ILIAS Bridge.',
        'path' => $path,
        'version' => BRIDGE_VERSION,
    ]);
} catch (Throwable $e) {
    ...
}

```